

Why Your Regex Stack Has a Thirty-Year Blind Spot

Joseph R. Lopez

May 2026

Contents

Your Regex Stack Has a Thirty-Year Blind Spot	1
One bypass class breaks WAFs, spam filters, secret scanners, and your new LLM guardrails. Five questions cut through every vendor pitch.	1
The 60-Second Version	1
§1 — The Strike: One Bypass, One Demo	2
§2 — Why This Isn't New: Same Algebra, Thirty Years of Whack-a-Mole	3
§3 — Is Your Stack Affected?	4
§4 — What To Do Monday	5
§5 — Five Questions to Ask Your Security Vendors	6
§6 — What We Measured	7
§7 — What We Do Not Claim	10
Appendix A — The Math, Compressed	10
Appendix B — Methodology, Compressed	11
References	12

Your Regex Stack Has a Thirty-Year Blind Spot

One bypass class breaks WAFs, spam filters, secret scanners, and your new LLM guardrails. Five questions cut through every vendor pitch.

Target Venue: Communications of the ACM (Practice) **Author:** Joseph R. Lopez **Word Count:** ~3,800 (body) + ~900 (appendices)

The 60-Second Version

If your stack has a layer that decides “block or allow” by matching patterns against text — your WAF, your AV signatures, your DLP, your spam filter, your secret scanner, your LLM guardrail — there is a bypass class that has worked against that layer’s design for thirty years and will keep working no matter how many patterns your vendor ships next quarter.

The bypass: insert an invisible or ignored character between every other character of the payload. The pattern engine sees no contiguous match. The downstream consumer — the LLM, the SQL engine, the human reading rendered output, the compiler — reassembles the filler away and acts on the original payload.

We tested this against 142 patterns from twelve LLM-guardrail libraries. Every single one is structurally vulnerable. We extended the test to ModSecurity’s flagship SQL-injection rules and SpamAssassin’s phrase rules: same bypass, same outcome. We have a mathematical proof, in a companion paper, that adding more patterns of the same kind cannot close the class.

The business consequence is not “buy a different product.” It is this: **the pattern of “cheap regex layer in front, expensive AI layer behind” has a serial veto problem.** When your regex layer says “looks fine,” your AI layer never sees the request. Your regex tier’s blindness becomes your stack’s blindness. This article shows you how to find that pattern in your architecture, what to ask your vendors, and what to budget — Monday.

§1 — The Strike: One Bypass, One Demo

A corporate chatbot has an input guardrail. The guardrail blocks **bomb**, **weapon**, **kill**, **attack**, and a few hundred other words. It is a regex matcher. Every user message routes through it before the language model sees a single token.

A user types this:

bomb

To the user’s eye, that says **bomb**. To the regex engine, the four characters between the **bs** are zero-width Unicode codepoints — invisible glyphs that take no visual space. The regex looks for the literal string **bomb** and finds nothing. The message passes. The language model decodes the zero-width characters as whitespace it can ignore and answers as though the user typed **bomb** directly.

That is the entire bypass. Two lines of Python construct the payload. The same construction works with NULL bytes, ASCII spaces, tabs, tildes, backticks, or any character outside the regex’s literal alphabet. We tested seven filler choices across four prime spacings — every-other, every-third, every-fifth, every-seventh character. All 392 combinations bypass.

Now substitute the version that maps to your stack:

- **Your WAF.** The payload is `SELECT * FROM users`, the filler is an SQL comment `/**/`. Your WAF finds no match. Your database receives `SEL/**/ECT * FROM users` and parses it as valid SQL. Your audit log shows nothing. **You inherited this exposure when you bought the appliance.**
- **Your spam filter.** The payload is `viagra`. The filler is a period. The text reads `v.i.a.g.r.a` to your filter and `viagra` to the executive who clicks it. **The phishing email that got through last month was this.**
- **Your secret scanner.** GitLeaks looks for AWS keys; the filler is a line continuation in a comment. Your scanner sees a broken token; your build pipeline sees the assembled secret. **The secret your post-mortem traced to a “missed scan” wasn’t missed — it was structurally invisible.**
- **Your LLM jailbreak detector.** The filler is a zero-width Unicode character. Your detector sees garbage; your model sees the attack. **The guardrail your CISO insisted you add does not see the payload your model will obey.**

Same construction. Same bypass class. Different filler character per generation of defense your

team bought. The next question is the one no vendor wants you to ask: *why has the same bypass been working against the same architectural pattern for thirty years?*

§2 — Why This Isn’t New: Same Algebra, Thirty Years of Whack-a-Mole

The defensive-security industry has been shipping pattern-matching defenses for three decades. AV signatures appeared in the late 1980s. WAFs in the mid-1990s. Network IDS at the end of that decade. Spam filters in the early 2000s. Code and secret scanners in the 2010s. LLM guardrails in the 2020s.

Every generation discovered the bypass. Every generation named it differently. Every generation’s vendors responded the same way: “we’ll add patterns for that.” That response has not worked, will not work, and cannot work — for a reason that has been sitting in a 1965 mathematics paper the entire time your industry has been billing for pattern-library updates.

The reason in plain English: a regex (or any equivalent substring matcher — trie, DFA, Bloom-filter blocklist) decides things by whether specific characters appear next to each other in specific orders. It cannot answer the question “does this text contain the payload if you read every other character?” It is the wrong shape of computation for that question. Adding more rules does not change the shape; the combination of two such matchers is itself a matcher of the same shape with the same blind spot.

Decade	What your defense was called	What the bypass was called	What this means for you
Late 1980s — 1990s	AV literal-string signatures	“no-op stuffing,” “byte interleaving”	The signature engine in your endpoint product carries this lineage in every literal-string rule it still runs
Mid-1990s — present	WAFs (ModSecurity, AWS WAF, Cloudflare)	“comment evasion,” “case mutation,” “Unicode normalization tricks”	The OWASP CRS rules behind your perimeter — including the SQLi rules we measured — bypass under the same construction today
Late 1990s — present	IDS/IPS regex tiers (Snort, Suricata)	“regex-tier evasion”	The narrow regex-matching tier in your detection stack inherits the bound; packet reassembly is a separate problem

Decade	What your defense was called	What the bypass was called	What this means for you
Early 2000s — present	Spam filters (SpamAssassin, regex content filters)	“acrostic spam,” “character substitution”	Your mail gateway’s phrase rules bypass the same way; we measured 9 of 9 production rules
2010s — present	Code and secret scanners (Semgrep, Bandit, GitLeaks, TruffleHog)	“comment-broken tokens,” “line-continuation splits”	The scanner gating your CI pipeline lets through any secret an attacker took ten seconds to interleave
2020s	LLM input/output guardrails (LLM-Guard, Presidio, Rebuff, Guardrails-AI, NeMo, OWASP-LLM)	“Bad Characters,” “payload splitting,” “encoding-based jailbreak”	The guardrail your AI team rolled out last quarter is the newest face of the oldest bypass on this list

A note on what is *not* in this table:

- Classical **polymorphic-virus** encoding (encryption + a self-mutating decryption stub) is a different attack class. The bound here applies to literal-string AV signatures bypassed by in-pattern interleaving, not to polymorphic encryption.
- Classical **IDS evasion** (Ptacek & Newsham 1998) — exploiting reassembly differences between the IDS and the endpoint — is a different attack class. The bound here applies to the IDS regex tier itself, not to packet-layer evasion.

We are precise about scope so the unification claim is verifiable, not rhetorical.

The point of the table is not “look how many things are broken.” The point is sharper: **your industry has been treating thirty years of one bypass class as thirty separate engineering problems.** Each vendor patches the specific filler character its customers report. The next attacker uses a different filler. Your renewal invoice arrives. The bound is structural; the patches you paid for are surface.

So the question becomes operational: how do you tell, before Monday’s standup, whether your stack carries this exposure?

§3 — Is Your Stack Affected?

Walk this checklist for each defensive layer in *your* stack. Answer “yes” to question 1 *and* “serial-AND” to question 5, and that layer is in the bypass class — and the bypass propagates to your whole system through it.

1. Does the layer make its block/allow decision by matching patterns (regex, signatures, blocklist) against the literal characters of the input? - Yes → continue - No (it uses a neural network, embedding-similarity, a fine-tuned classifier, an LLM-as-judge, behavioral

analysis, rate limiting, or a parser that fully normalizes input first) → out of scope; you are fine on this axis

2. Are the patterns substring-matching — i.e., do they look for the payload anywhere in the input rather than requiring the input to start or end with the pattern? - Yes (the common case) → continue - No (rare; the layer requires exact full-string match) → still vulnerable, but the attack surface narrows for you

3. Does the input pass through the layer *before* reaching your more-capable downstream system (LLM, parser, classifier, human reviewer)? - Yes → the bypass reaches your downstream system - No (only logged, not blocked) → the bypass is a detection-evasion problem for you, not a control-bypass problem

4. Does your downstream system “decode” the bypass — i.e., does it ignore or normalize the filler character the attacker is likely to use? - An LLM downstream → yes, decodes reliably (we measured 91.7% across four open-weight models) - A human reading rendered output → yes, decodes visually (zero-width characters are invisible) - A SQL parser → sometimes (whitespace and comment fillers decode; NULL bytes typically don't) - A compiler reading scanned source → yes, strips comments and whitespace before token assembly - A schema validator with no further consumer → no, processes literally

5. What is your layer's relationship to your more-capable defenses (neural classifier, LLM judge, deep parser, manual review)? - **Serial-AND** — your regex tier blocks first; only ambiguous traffic reaches the next tier → **your regex tier's blindness becomes your stack's blindness** - **Parallel-OR** — every request reaches both tiers; either tier can block → your more-capable tier's capacity survives - **No more-capable tier exists** — your regex layer is the only defense → maximum exposure on your roadmap

The serial-AND configuration is overwhelmingly the default in production because it is what makes the cheap regex tier financially worthwhile to your CFO. It is also exactly the configuration in which the bypass propagates straight through your stack. Most teams discover their architecture is more serial than the diagram their vendor showed them.

If you found yourself in serial-AND, the next question is what you do about it — without buying a new product or rewriting your security organization from scratch.

§4 — What To Do Monday

Four moves, in priority order. One engineer-week for an LLM-stack deployment; longer if you carry full WAF + IDS + AV + DLP coverage.

1. Map every pattern-matching tier in your stack and its composition with downstream tiers. For each: input scanner, output scanner, tool-call validator, schema validator, retrieval-content scanner, WAF rule set, IDS signature set, secret/code scanner, spam-filter rule set. Write down whether it is serial-AND or parallel-OR with the next tier. Most teams discover they are more serial than they thought; once you have it on paper, the cost difference between “every request reaches the expensive tier” and “regex first” is concrete and budget-able. **This is the diagram your auditor will ask for in twelve months and you will not want to draw under deadline.**

2. Reframe what your regex tier is actually for. Substring matchers are excellent at exact-keyword blocks, format validators (credit-card-shaped strings), audit logging of contiguous patterns, and high-throughput cheap filtering of obviously-malformed traffic. They are structurally bad at the bypass class. Tell your security team and your vendors which job each tier is doing, and stop charging your regex tier with work it cannot do. **The regex you wrote at 2 AM during the last incident is good at what you wrote it for; it is not your jailbreak detector.**

3. Where bypass cost is high, route the bypass class through a more-capable arbiter without your regex tier vetoing it. In increasing order of cost: a transformer-based safety classifier in parallel with your regex tier; an LLM-as-judge for ambiguous traffic; a normalizing parser that strips known filler classes before the regex check (closes specific filler choices, not the class); for the highest-stakes cases, an architecture in which sensitive secrets and context never reach the inference layer at all (an MIT-licensed reference implementation accompanies this work).¹

4. Put the bypass class in your threat model as a known-unknown. Vendor pattern libraries cover what they cover. The math says they cannot cover the bypass class within the substring-matching design. “*Yes, our neural tier handles encoded attacks*” is a posture you can defend in your next board update; “*yes, our regex patterns catch encoded attacks*” is a claim no vendor can support for this attack class — and your name is on the risk register, not theirs. Update your contractual language and your audit responses accordingly. **The SOC 2 question you couldn’t answer last year has an answer now: it is the architecture, not the rule count.**

These four moves are inside your authority. The fifth move requires conversations with the people selling you the regex tier — and those conversations need a script.

§5 — Five Questions to Ask Your Security Vendors

Five questions. The structure of the bypass is the same whether you are buying a WAF, an LLM guardrail, an IDS, a secret scanner, or a DLP product, so the structure of the questions is the same. Ask them on the next renewal call. Watch which way the vendor flinches.

1. “What is the architectural pattern in front of and behind your pattern-matching tier? Is it serial-AND or parallel-OR with the next layer? If serial-AND, what is your answer to the modular-counting bypass class?” A vendor who cannot describe the composition pattern, or who cannot describe the bypass class, is selling you a black box. You are now the only adult in the room who can describe it. That asymmetry is your leverage.

2. “What input normalization runs before your pattern match? Specifically: do you strip zero-width Unicode characters, normalize Unicode (NFC or NFKC), strip NULL bytes, collapse whitespace, decode comments?” Each of these closes specific filler choices. None closes the class. We surveyed four widely-deployed LLM-guardrail libraries; none performed Unicode normalization, and only one stripped some filler characters as a side effect of broader sanitization. If your vendor’s answer is “we strip the zero-width characters,” your next question is “and the NULL bytes? And the tildes? And the next codepoint the attacker tries?”

3. “Show me your test corpus for the modular-counting bypass class. Specifically:

¹*Disclosure: this implementation was developed by the author as part of this project. Verify its properties against your own threat model before deployment.*

at what spacings (every-other, every-third, every-fifth) and with which fillers do you test, and against what patterns?” A vendor with a real test corpus will hand you numbers. A vendor without one will pivot to “our customers haven’t reported this.” That pivot is the answer: the customers haven’t reported it because the customers don’t see it. You don’t see it either, until you run the test we describe in Appendix B.

4. “When your regex tier passes traffic through, does the more-capable tier behind it see the original input or your regex tier’s normalized version?” If the more-capable tier only sees the regex tier’s normalized input, your regex tier’s blind spots transfer to the more-capable tier even in deployments your vendor’s diagram labeled “parallel.” You have to ask explicitly. The diagram will not save you.

5. “What is your roadmap for moving the bypass class out of the substring-matching tier?” This is the diagnostic question — the one whose answer tells you whether you are buying a serious product or a patch subscription. A vendor with a serious answer will talk about parallel composition, transformer-based classifiers, normalizing parsers, or architectural changes that route sensitive content out of the inference path. A vendor without one will offer to add patterns. **You now know what “we’ll add patterns for that” means. Forward this article to your procurement lead before the next renewal cycle closes.**

The questions are calibrated to the vendor pitch. The next section is calibrated to your own measurement.

§6 — What We Measured

Five empirical measurements, plus extension to two non-LLM defensive classes. Methodology and per-pilot detail in **Appendix B**; full artifact JSONs are in the project repository so you can rerun every cell against your own corpus.

#	What we measured	Sample	Result	What this means for you
1	How many production guardrail patterns are structurally vulnerable	142 patterns from 12 LLM-guardrail sources (9 third-party libraries, 3 author-assembled sets, 30%)	142 / 142 vulnerable	Every pattern in the corpus has the structural blind spot — not because of bad pattern design, but because the substring-matching design <i>forces</i> the blind spot into every rule your vendor ships

#	What we measured	Sample	Result	What this means for you
2	Does the basic bypass work on a sample of representative patterns	14 (payload, pattern) test cases $\times$ 4 spacings (every-other, every-third, every-fifth, every-seventh), NULL-byte filler	56 / 56 bypass	The bypass works at every prime spacing the math predicts; your “we block NULL” mitigation cannot save you
3	Does it still work if your vendor blocks NULL bytes	Same 14 cases $\times$ 4 spacings $\times$ 7 filler characters (NULL, ASCII space, tab, tilde, backtick, two zero-width Unicode codepoints)	392 / 392 bypass	Patching one filler does not close the class. There are always more filler characters than patches in your vendor’s release notes
4	Does it work against patterns shipped by named third-party vendors	100 library-shipped patterns; 48 baseline-matchable by our payload synthesizer (across 8 of 9 vendors)	48 / 48 bypass under both NULL and ASCII-space filler	Shipped patterns from named vendors bypass at the same rate as test patterns; your procurement diligence cannot filter for this
5	Do downstream LLMs reliably reconstruct the encoded payload (and is this just a response-prior artifact)	4 open-weight models (3B–22B) $\times$ 3 fillers $\times$ 5 benign payloads = 60 decode cells; 20 random-character control cells	55 / 60 decode (91.7%); 0 / 20 control (0%); +91.7-pp lift	Modern LLMs reliably decode the bypass; the gap is not a coincidence of the model liking the topic. The model your stack ships in 2026 will decode it harder, not less

#	What we measured	Sample	Result	What this means for you
6	Does the bypass extend to non-LLM defensive classes	6 ModSecurity OWASP CRS 4.27 SQL-injection patterns + 6 SpamAssassin 20_phrases.cf rules; 20 synthesized payloads; NULL, ASCII-space, U+200B fillers	WAF (ModSec): 10/10 baseline-matched; 8/10 NULL-bypass, 10/10 ASCII-space, 10/10 zero-width-space. Spam (SpamAssassin): 9/10 baseline-matched; 9/9 NULL, 9/9 ASCII-space, 9/9 zero-width-space. The two NULL “failures” are SQL-comment patterns whose alphabet includes NULL by design — they bypass under printable-space filler instead.	The bypass works against the production WAF and spam-filter rules in your stack right now, in exactly the way the math predicts: no single filler closes the class

We additionally surveyed the source of four production guardrails (LLM-Guard `BanSubstrings`, Presidio analyzer, Rebuff heuristic detector, NeMo Guardrails content-safety check) for input normalization. None perform Unicode normalization; only one strips zero-width Unicode codepoints as a side effect of broader sanitization, and printable-filler bypass remains intact against it.

The remaining defensive classes named in §2 (literal-string AV signatures, IDS regex tiers, code/secret scanners) are by mathematical argument rather than direct empirical pilot in this release. We expect the same outcome because the structure is identical; direct empirical extension is a clean follow-up — and one we will run, but one you can also run yourself with the artifacts cited above.

What we measured tells you what the bypass *does*. What we did not measure tells you the size of the perimeter — and that perimeter has edges, which honest engineering requires us to mark.

§7 — What We Do Not Claim

We do not claim that all AI security is mathematically impossible. We do not claim that all defensive security is mathematically impossible. We do not claim neural guardrails or LLM-judges share this blind spot — they do not, because they are a different and strictly more capable class of computation.

The bound is narrow, sharp, and unconditional inside its scope: it applies to substring-matching pattern recognizers (regex, trie, DFA, finite-language Bloom filter) and to any Boolean composition of them, including multi-tier compositions across product boundaries. It does not apply to embedding-based similarity filters, fine-tuned safety classifiers, model-internal safety training (RLHF, constitutional AI, refusal training), behavioral or rate-limit guardrails, or anomaly detection on edit-distance or dynamic-programming features.

We do not claim our LLM-guardrail measurements generalize universally to every non-LLM defensive class named in §2. We *do* present empirical extension to two non-LLM classes (ModSecurity and SpamAssassin) — the bypass succeeds across all baseline-matched payloads in both. The remaining classes are by argument; the empirical sweep across them is a follow-up.

The original framing of this work was a much larger universal-impossibility claim covering all AI systems with compression capability. That claim turned out to be false; a single counterexample destroys it, and we document that destruction publicly alongside the project repository. The bound that survives — the one this article describes — is narrower and unconditional within its scope. We mention this because *integrity about which claims survived adversarial review* is itself a posture this field, and the post-mortem culture you operate inside, needs more of.

The math is in **Appendix A** if you want to read it. The architecture review is yours.

Appendix A — The Math, Compressed

The full development is in the companion technical paper at `paper/main.tex` (28 pages, arXiv preprint). The compressed version:

A regex (and equivalently any trie, DFA, or finite-literal-set Bloom filter) that scans for substring matches falls into a class of computation formal-language theorists call **star-free**. Schützenberger (1965) proved that star-free languages are exactly those whose **syntactic monoid** — an algebraic object that captures what the recognizer can distinguish — has no nontrivial cyclic subgroups. In plainer words: these recognizers cannot count modulo a prime.

Barrington, Compton, Straubing, and Thérien (1992) connected this to circuit complexity: every star-free language is decidable in $\mathbf{AC}^0$, the constant-depth, polynomial-size, unbounded-fan-in Boolean circuit class. Furst, Saxe, and Sipser (1981) and Håstad (1987) proved that $\mathbf{AC}^0$ provably cannot compute the parity function — the simplest case of modular counting.

The bypass is the constructive witness of this gap. Place the payload at every other character; place a filler character (one outside the pattern’s literal alphabet, or one whose effect on the shifted alphabet is captured by a separate “Regime 2” argument we develop in the companion paper) at the alternating positions. The substring matcher cannot align with the payload because doing so would require it to count positions modulo 2. No combination of additional star-free patterns closes the class, because the union of star-free languages is star-free, and the syntactic monoid of

a Boolean combination divides the direct product of the components’ monoids (Pin 1986). This is why “add more patterns” cannot work: *any* additional pattern of the same kind, at any tier of the pipeline, leaves the system in AC^0 .

A neural classifier (any transformer-based model) operates in TC^0 , a strictly larger class that does include modular counting as a capacity bound (Merrill & Sabharwal 2023; Chiang, Cholak, Pillay 2023). Whether a *specific* trained transformer reliably realizes this capacity is a separate empirical question (Hahn 2020 raised concerns about parity at unboundedly long inputs); but the architecture-level capacity gap is the structural reason the composition trap matters: a TC^0 tier behind an AC^0 veto inherits the AC^0 blindness; a TC^0 tier in parallel with an AC^0 tier does not.

The full proof of the bound for both regimes (alphabet $\subsetneq \Sigma$; alphabet $= \Sigma$), the word-boundary-anchor lemma, and the closure argument under composition appear in `paper/main.tex` §§3–10.

Appendix B — Methodology, Compressed

Full methodology and per-pilot artifacts are in the project repository. The compressed version:

Corpus. 142 patterns from twelve sources. Nine third-party LLM-guardrail libraries contribute 100 patterns: LLM-Guard (12), llm-guard-py (12), Rebuff (10), Guardrails-AI (12), Presidio (15), GitLeaks (8), LangKit (10), BodAIGuard (10), and the regex subset of NeMo Guardrails (11). Three author-assembled sets contribute 42 patterns (29.6% of the corpus): 8 inspired by OWASP Top 10 for LLMs, 15 from a curated WAF-generic pattern set, 19 author-constructed adversarial test patterns. Released as `corpus_full.csv`.

Vulnerability verification. Each pattern compiled to Thompson NFA $\rightarrow$ minimal DFA (Hopcroft minimization) $\rightarrow$ transition monoid enumerated by BFS over the Cayley graph $\rightarrow$ tested for $x^n = x^{(n+1)}$ at bound $n \leq |M|^2$ (Pin 1986). 119 verified by completed enumeration; 22 timed out within a grammar covered by the substring-aperiodicity lemma (theorem-implied vulnerable); 1 (the Presidio IBAN regex) verified vulnerable by a separate word-boundary-anchor lemma.

Pilot A — primary bypass. 11 author-assembled patterns $\times$ 4 prime spacings (2, 3, 5, 7) $\times$ 12 payloads, NULL-byte filler. Artifact: `results/mod_p_bypass_matrix.json`.

Pilot B — filler diversity. Same 11 patterns $\times$ 4 primes $\times$ 7 fillers (NULL, ASCII space, tab, tilde, backtick, U+200B, U+200C). 392 tests, all bypass. Artifact: `results/printable_filler_bypass.json`.

Pilot C — LLM decode reliability with control. Four open-weight models (llama3.1:8b, llama3.2:3b, qwen2.5-coder:14b, codestral; 3B–22B parameters) $\times$ 3 fillers $\times$ 5 benign payloads = 60 decode cells; plus 20 random-character control cells to rule out response-prior artifacts. Decode 55/60 (91.7%); control 0/20 (0%); lift +91.7pp. Artifact: `results/llm_decode_pilot_v2.json`.

Pilot D — third-party library patterns. Literal-glue payload synthesizer applied to 100 library-shipped patterns; 48 baseline-matched across 8 of 9 libraries (Presidio’s PII patterns are the synthesis exception). 48/48 bypass under both NULL and ASCII-space filler. Artifact: `results/library_pattern_bypass.json`.

Pilot E — non-LLM defensive classes. 6 ModSecurity OWASP CRS 4.27 SQL-injection patterns + 6 SpamAssassin `20_phrases.cf` patterns; 20 synthesized payloads; MOD_2 bypass

tested with NULL, ASCII-space, and U+200B fillers. Per-class results in §6 row 6. Artifact: `results/non_llm_defense_bypass.json`.

Library normalization survey. Source-level inspection of LLM-Guard BanSubstrings, Presidio analyzer, Rebuff heuristic, and NeMo Guardrails content-safety check on `main/develop` HEAD. None perform Unicode normalization (NFC/NFKC) or strip zero-width Unicode codepoints prior to regex match. Rebuff strips NULL bytes and zero-width codepoints as a side effect of broader sanitization; printable-filler bypass remains intact. Artifact: `results/library_normalization_survey.md`.

Pilots not run, in scope of the bound. Output-side DLP regex; tool-call argument validator regex; JSON Schema `pattern` field validators; trie/DFA/Bloom-filter blocklists; literal-string AV signatures; IDS regex tiers; code/secret scanners; multi-tier vulnerable pipelines. The mathematical bound applies; direct empirical pilots are clean follow-ups.

References

1. Schützenberger, M.-P. (1965). On finite monoids having only trivial subgroups. *Information and Control* 8(2), 190–194.
2. McNaughton, R., and Papert, S. (1971). *Counter-Free Automata*. MIT Press.
3. Barrington, D. A. M., Compton, K., Straubing, H., and Thérien, D. (1992). Regular languages in NC¹. *Journal of Computer and System Sciences* 44(3), 478–499.
4. Furst, M., Saxe, J. B., and Sipser, M. (1981). Parity, circuits, and the polynomial-time hierarchy. In *Proc. 22nd IEEE FOCS*, 260–270.
5. Håstad, J. (1987). *Computational Limitations of Small-Depth Circuits*. PhD thesis, MIT.
6. Krohn, K., and Rhodes, J. (1965). Algebraic theory of machines I: Prime decomposition theorem for finite semigroups and machines. *Trans. AMS* 116, 450–464.
7. Pin, J.-E. (1986). *Varieties of Formal Languages*. Plenum Press.
8. Boucher, N., Shumailov, I., Anderson, R., and Papernot, N. (2022). Bad Characters: Imperceptible NLP Attacks. In *Proc. IEEE Symposium on Security and Privacy*.
9. Wei, A., Haghtalab, N., and Steinhardt, J. (2023). Jailbroken: How does LLM safety training fail? In *NeurIPS*.
10. Hackett, W., Birch, L., Trawicki, S., Suri, N., and Garraghan, P. (2025). Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. In *Proc. First Workshop on LLM Security (LLMSEC), co-located with ACL 2025*.
11. Li, B., Xing, H., Tian, C., Huang, C., Qian, J., Xiao, H., and Feng, L. (2024). Structural-Sleight: Automated jailbreak attacks on large language models utilizing uncommon text-organization structures. arXiv preprint 2406.08754.
12. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proc. Workshop on Artificial Intelligence and Security (AISec)*.
13. Merrill, W., and Sabharwal, A. (2023). The parallelism tradeoff: Limitations of log-precision transformers. *TACL* 11, 531–545.
14. Chiang, D., Cholak, P., and Pillay, A. (2023). Tighter bounds on the expressivity of transformer encoders. In *Proc. 40th International Conference on Machine Learning (ICML)*, PMLR vol. 202.

15. Hahn, M. (2020). Theoretical limitations of self-attention in neural sequence models. *TACL* 8, 156–171.
16. Ptacek, T. H., and Newsham, T. N. (1998). Insertion, evasion, and denial of service: Eluding network intrusion detection. Technical Report, Secure Networks, Inc.
17. Liu, C., and Stamm, S. (2007). Fighting unicode-obfuscated spam. In *Proc. Anti-Phishing Working Groups 2nd Annual eCrime Researchers Summit*, 45–59.
18. 0xInfection (curator) (2024). Awesome-WAF: A curated list of Web Application Firewall bypass techniques. <https://github.com/0xInfection/Awesome-WAF>
19. Project repository (this work, 2026), to be deposited at a stable DOI for camera-ready. Includes: 142-pattern corpus (`corpus_full.csv`), monoid enumeration scripts, primary bypass artifact (`mod_p_bypass_matrix.json`), four follow-up empirical pilots (`printable_filler_bypass.json`, `library_pattern_bypass.json`, `llm_decode_pilot_v2.json`, `non_llm_defense_bypass.json`), four-library normalization survey (`library_normalization_survey.md`), formal proofs (`paper/main.tex`), the parity-projection counterexample referenced in §7 (`counterexamples.md`), and the secrets-router reference implementation.